

Operating System Access

The `sh` and `!` verbs, the list that decides who may use them, and why the list is keyed to the person rather than the account.

`sh` runs a Windows command from the SD prompt. `!` is the same verb under a shorter name. **They are the only way out of SD to the operating system from TCL**, and who may use them is decided by a file in the system account rather than by an account's tier.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a user-set page is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, from an unelevated session in an account that is on the list.

The two verbs

```
sh command
! command
```

Everything after the verb is handed to the shell as typed:

```
:sh echo hello-from-the-shell
hello-from-the-shell
:! echo via-the-bang-form
via-the-bang-form
```

Never type `sh` with nothing after it in a script. The configured shell for the bare form is interactive — `config` reports it as

```
SH      C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NoLogo
SH1     C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NonInteractive
```

so a bare `sh` in a piped session, a phantom or a scheduled job hands control to a shell with nobody at the keyboard and **waits for ever**. The `SH1` form, which is what `sh command` uses, is `-NonInteractive`.

The shell you get is Windows PowerShell 5.1

Not `pwsh`, and the difference bites:

```
:sh echo delta && echo epsilon
At line:1 char:12
+ echo delta && echo epsilon
+           ~~
The token '&&' is not a valid statement separator in this version.
```

`&&` and `||` **do not exist there**. Nor do the ternary, null-coalescing or null-conditional operators. Chain with `;`, and test with `if ($?) { ... }`.

Pipes and redirection do work, for an account that is on the list:

```
:sh echo alpha-beta | findstr alpha
alpha-beta
:sh echo gamma > zzsh.txt
:sh Get-Content zzsh.txt
gamma
```

The second and third lines are separate `sh` invocations, so **the working directory persists between them** — the file written by one was read by the next.

SD refuses to nest

```
:sh sd
SD is already running in this session - type EXIT to return to it.
```

The child shell is marked, and SD checks the mark:

```
:sh Get-ChildItem Env:SD_SESSION
Name                               Value
```

SD_SESSION

1

That is worth knowing when writing a script for `sh` to run — anything it invokes inherits `SD_SESSION`, so a script that starts SD as part of its work will be refused, wherever it is called from.

Who is allowed: `os.users`

The permission is a record in a file in the system account, not a tier. `sdsys/os.users` holds one record per person:

record id the **Windows login name**

field 1 `yes` to allow `sh` and `!`

field 2 `yes` to allow `OS.EXECUTE` from a program, and the screen editors

Anything other than `yes` means no, and **a missing file or a missing record means no**. That is the opposite of the tier lists, where a missing record means the full set — do not carry the convention across.

It is keyed to `@logname`, the person, not the account. The permission therefore does not change when somebody `logto`s somewhere else. That is deliberate: the question *may this person reach the operating system* has one answer per person, and an account they can enter should not be able to change it.

EDITING IT

`os.users` is an ordinary SD file in the system account, edited with `ed` from `SDSYS` — which needs an elevated session to enter. `create.account` **writes the record for an administrator account as it creates it**, with both fields `yes`, and `modify.account`'s `sh-on`/`sh-off`/`os-on`/`os-off` keywords set the two fields afterwards. Hand-editing remains the only route for the case those keywords refuse.

The file's ACL is the whole of the protection. `os.users` is read-only to `sdusers` on disk, which is what stops somebody adding their own name to the list from inside their own account. The command processor reads it in the user's own process. **Without that ACL this control is decoration** — if you are hardening an installation, check it rather than assume it.

The three outcomes

The gate and the metacharacter rule are separate tests, and they combine like this:

	PLAIN COMMAND	PIPES, REDIRECTION, CHAINING
on the list	runs	runs
not listed, elevated	runs	refused 5240
not listed, unelevated	refused 10053	refused 10053

```
don is not permitted to use the operating system shell
```

is message 10053, and it names the person rather than the account.

The middle row is the one people misread. An elevated session that is not on the list keeps a restricted shell: it may run a command, but not one containing shell metacharacters. **Being on the list is what buys a real shell** — pipes, redirection and chaining are most of what a programmer wants one for, and that was the ruling behind lifting the ban for listed accounts. Elevation on its own does not lift it.

What this does not gate

A program's `OS.EXECUTE` is not on this path. `OS.EXECUTE` from SD BASIC compiles to a different opcode and is governed by field **2** of the same record, not by field 1 and not by the metacharacter test. So the two halves are set independently, and `sh-off` deliberately leaves `OS.EXECUTE` alone.

The form that does go through this gate is `execute 'sh ...'` from a program, because that runs the TCL verb.

The screen editors read field 2, not field 1 — an account can be refused `sh` and still run `edit` and `micro`, and the reverse.

Who has these verbs

`sh` and `!` are administrator-tier, so an ordinary account does not have the names. **And the tier is not the permission** — an administrator account whose Windows login is not in `os.users`, and whose session is not elevated, has the verb and is refused by it. **Two gates, and both must pass.**

An SD administrator is a shell on this machine

State this plainly to anyone deciding who gets an administrator account:

An SD administrator can run operating-system commands on the server as LocalSystem — from this machine. A sign-in from any other computer is refused, over ssh and over the API alike.

The local half is not a defect, and no single setting produces it. It follows from three rules that are each reasonable on their own:

An administrator always has API access	and it cannot be taken away
An administrator always has <code>OS.EXECUTE</code>	and that cannot be taken away either
For a session that arrived over a socket, <code>os.users</code> is the authority	the session's own token is LocalSystem

So an administrator account is, in effect, an operating-system shell on the server, and the operating system reports such a session as `nt authority\system`.

WHY REMOTE IS SHUT

Those three rules would have made the administrator tier a shell for *anyone who could reach the port*, from anywhere. SD closes that at the door instead of weakening any of the three: **administration requires a session Windows can show a consent prompt on**, which means the console, or a remote-desktop or remote-control product installed as a service — not ssh and not the API.

The refusal comes *after* the password has been checked, so it is a refusal rather than a silent drop, and it says what it is:

An administrator may not sign in to this machine from another one.

Only the **tier** is refused. An ordinary or programmer account reaches the same machine over the same route and gets a session as before, which is worth knowing when you are diagnosing a connection that failed: if a non-administrator can get in, the network and the listener are fine.

THE ONE ROUTE THIS DOES NOT CLOSE

An ssh tunnel ends on the server, so a connection forwarded through one arrives looking local, and is admitted. SD sees the address the connection came from, and a tunnelled connection genuinely comes from this machine.

That is a real limit and it is stated here rather than glossed over. Building such a tunnel still needs an ssh login to Windows in the first place, so it is not open to a stranger — but if your threat model includes an administrator who should not be administering remotely, close port forwarding in `sshd_config` rather than relying on this gate.

What limits the rest of it is who holds an administrator account, who holds a credential for one, and who can reach the console.

The verify suite asserts all of this — the local session working, the remote one refused, and an ordinary account admitted over that same remote route as the control — so a future change that quietly reopened it shows up as a failing test rather than as a page that had silently become false.

The first logto out of the system account ends elevation

An administrator working in `SDSYS` is elevated. **The moment they `logto` anywhere else, that elevation is given up** — the administrator flag is cleared and the elevated helper is stopped.

This is deliberate. Without it the helper would outlive the rights it belongs to, leaving a session able to do privileged work from an ordinary account.

The consequence a reader meets first is that **a second hop is refused**:

```
:logto sales
:logto payroll
```

The second `logto` is not running as an administrator any more, so it costs a fresh UAC prompt. Recover by going back:

```
:logto sdsys
```

An administrator who chains two hops and does not know this will read it as a fault. It is the design working.

The step is quiet for the ordinary case: a session moving between two ordinary accounts never had privilege, so nothing is given up and nothing is written to the audit trail.

See also

Accounts and Security · Sessions and Locks · Remote Access and the Machine.